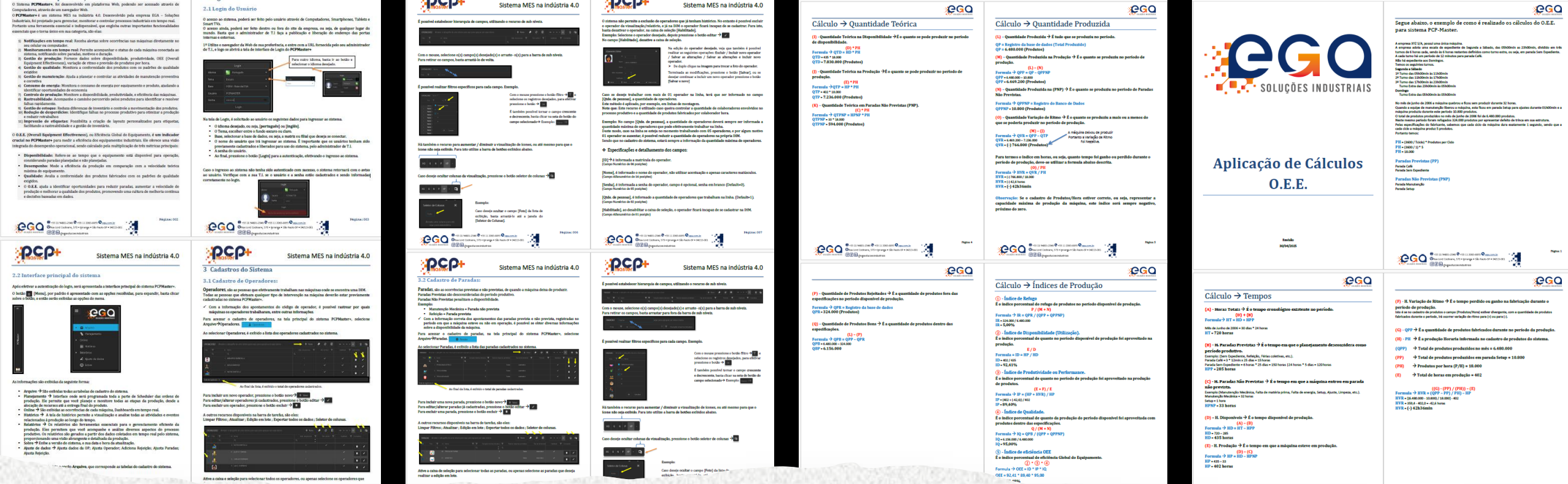


Modelo de IA Personalizado com RAG

Projeto Integrador 6

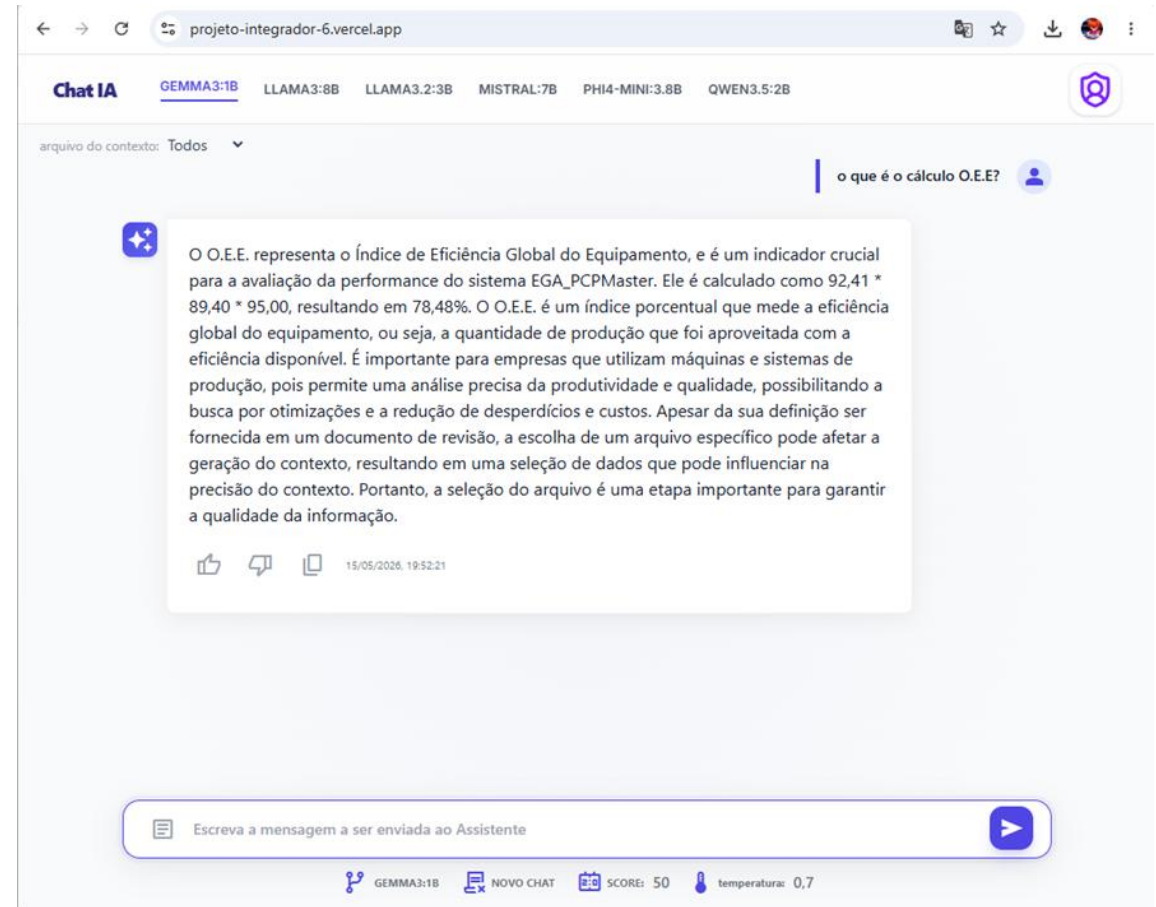


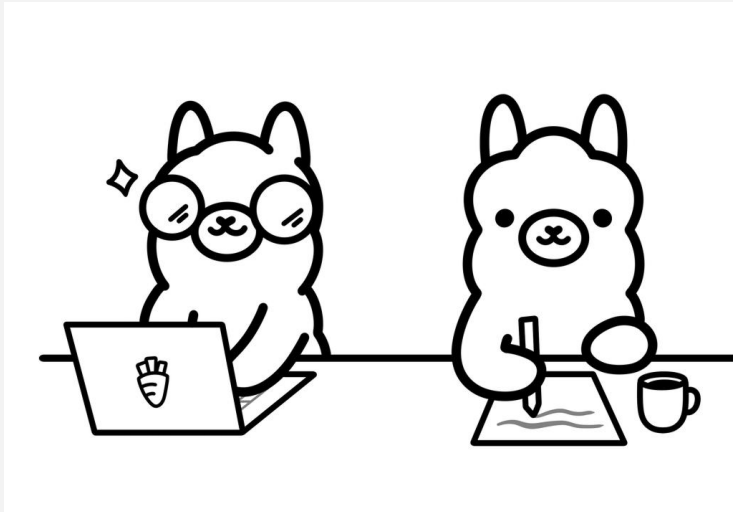
Documentos da EGA

Soluções Industriais

O Sistema

- Permite extrair conhecimento dos documentos
- Fundamentados em dados reais criados pela própria empresa
- Resultados rápidos, claros e precisos
- Acesso interno e externo disponibilizados por APIs, site web e app mobile
- <https://projeto-integrador-6.vercel.app/>





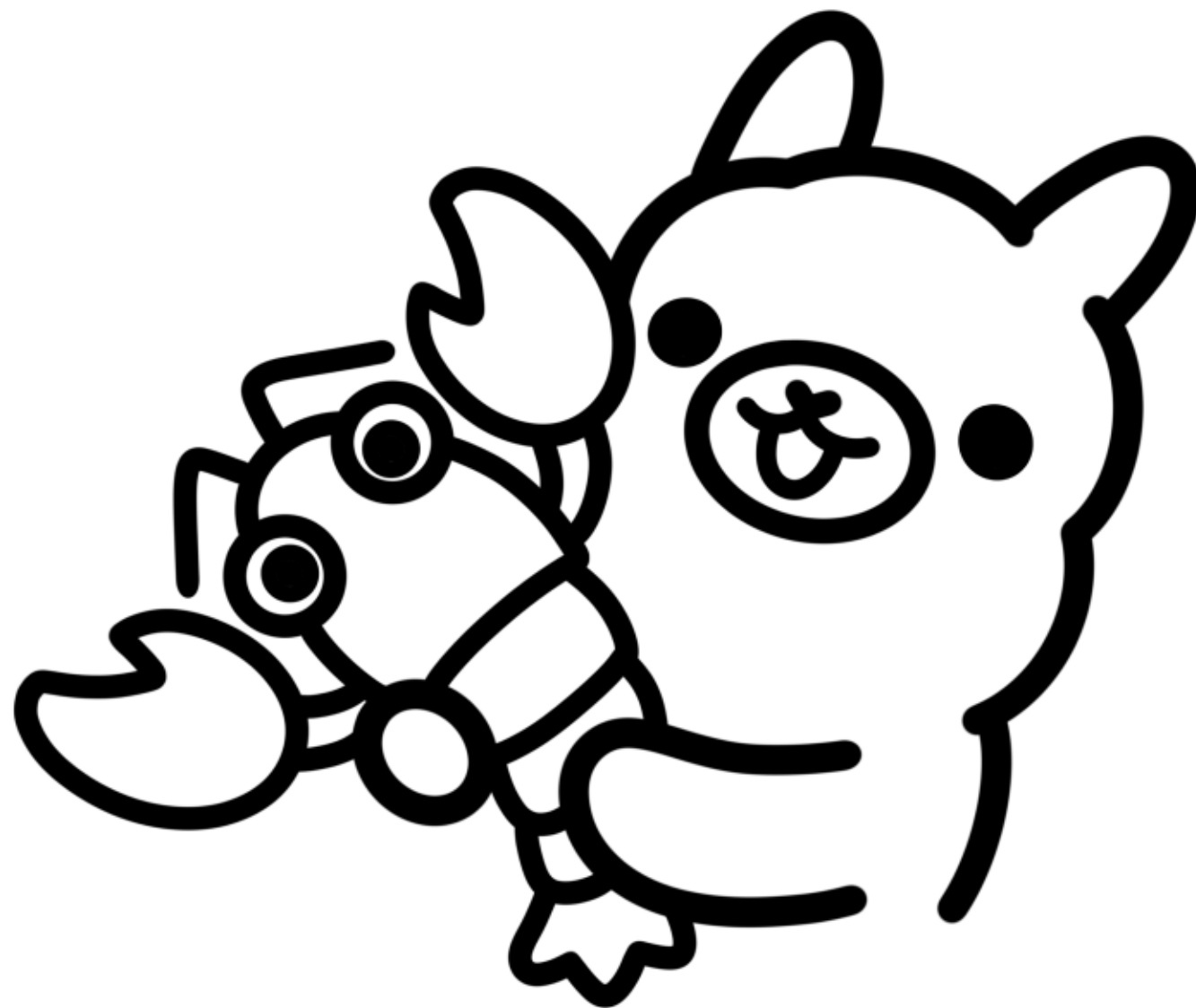
Ollama

- É uma ferramenta de código aberto que executa LLMs (*Large Language Models*)
- Permite executar de forma muito fácil modelos localmente, via CLI, GUI ou APIs
- É multiplataforma, podendo operar sem internet, garantindo privacidade de dados e eliminando custos com APIs de nuvem
- <https://ollama.com/>

Alguns modelos

(dentre os milhares)

- Gemma – Google
 - Phi – Microsoft
 - Llama – Meta
 - DeepSeek – DeepSeek
 - Mistral – Mistral AI
 - Qwen – Alibaba
-
- Features: Cloud, Embedding, Vision, Tools, Thinking



> ollama run gemma3:1b

> por que o céu é azul?

> A cor azul do céu é um fenômeno fascinante que se deve a um processo chamado **espalhamento de Rayleigh**. Aqui está uma explicação detalhada:

> **1. A Luz do Sol e as Cores:**

> * A luz do sol que chega à Terra é composta por todas as cores do arco-íris. Cada cor tem um comprimento de onda diferente.

> * O vermelho e o laranja têm comprimentos de onda maiores e podem atravessar a atmosfera com mais facilidade.

> * As cores mais leves, como o azul e o verde, têm comprimentos de onda menores e são dispersas mais facilmente

> pela atmosfera.

> **2. O Espalhamento de Rayleigh:**

> * O ar ao redor da Terra não é homogêneo. Ele é composto por moléculas de nitrogênio e oxigênio.

> ...

Teste Ollama

```
root@baccfac9f29:/ # ollama run gemma3:1b
>>> por que o céu é azul?
A cor azul do céu é um fenômeno fascinante que se deve a um processo chamado **espalhamento de Rayleigh**. Aqui está uma explicação detalhada:

**1. A Luz do Sol e as Cores:**

* A luz do sol que chega à Terra é composta por todas as cores do arco-íris. Cada cor tem um comprimento de onda diferente.
* O vermelho e o laranja têm comprimentos de onda maiores e podem atravessar a atmosfera com mais facilidade.
* As cores mais leves, como o azul e o verde, têm comprimentos de onda menores e são dispersas mais facilmente pela atmosfera.

**2. O Espalhamento de Rayleigh:**

* O ar ao redor da Terra não é homogêneo. Ele é composto por moléculas de nitrogênio e oxigênio.
* Quando a luz do sol entra na atmosfera, ela colide com essas moléculas. Esse processo faz com que a luz se espalhe em diferentes direções.
* O espalhamento de Rayleigh é mais eficaz em comprimentos de onda menores, ou seja, na coloração azul e verde. É por isso que o céu parece azul.

**3. Por que o céu é azul durante o dia?>>>

* Durante o dia, a luz do sol atinge nossos olhos em um ângulo relativamente pequeno em relação à nossa perspectiva. Isso faz com que a luz azul seja espalhada em todas as direções mais intensamente.
* Como a luz azul é espalhada ao redor, ela ilumina o céu e o torna azul.

**4. Por que o pôr do sol é vermelho/laranja?>>>

* Quando o sol está baixo no horizonte, a luz do sol precisa viajar por uma distância maior através da
```

Resultado de testes de alguns modelos no ajuste de prompts

modelo	tamanho	parâmetro	tempo	comprimento	opnião
gemma2:2b	1.52GB	2.6B	15	797	inglês
gemma3:1b	0.76GB	999.89M	41	586	inglês
gemma3:4b	3.11GB	4.3B	171	868	ok
phi3.5:3.8b	2.03GB	3.8B	543	5738	alucinou
phi4-mini:3.8b	2.32GB	3.8B	234	1469	ótimo
qwen3:0.6b	0.49GB	751.63M	98	481	ok
qwen3:1.7b	1.27GB	2.0B	158	961	médio
qwen3:4b	2.33GB	4.0B	511	1410	bom
qwen3.5:2b	2.55GB	2.3B	515	626	ok
qwen3.5:4b	3.16GB	4.7B	1320	1059	ok
llama3:8b	4.34GB	8.0B	486	794	ok
llama3.2:3b	1.88GB	3.2B			
mistral:7b	4.07GB	7.2B	509	1767	inglês
llm2.5-thinking:1.2b	0.68GB	1.2B	69	611	inglês
gemma3:270m	0.27GB	268.10M	25	244	ruim
llama2:7b	3.56GB	7B	563	1600	inglês

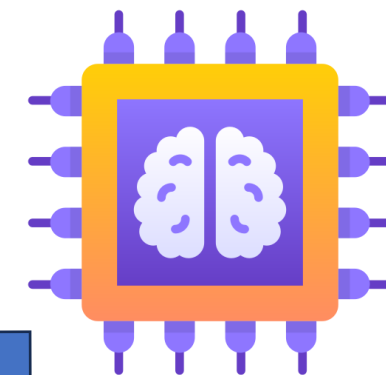
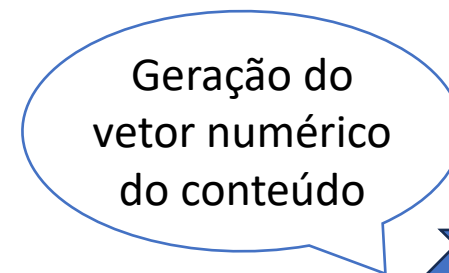
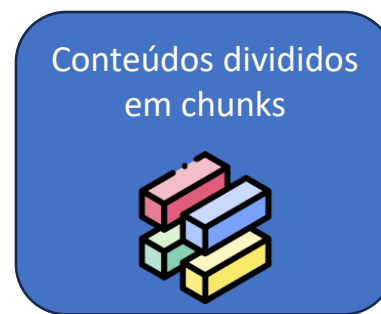
modelo	tamanho	parâmetro	tempo	comprimento	opnião
gemma2:2b	1.52GB	2.6B	307	929	ok
gemma3:1b	0.76GB	999.89M	347	651	simples
gemma3:4b	3.11GB	4.3B	988	1176	bom
phi3.5:3.8b	2.03GB	3.8B	1199	1104	bom
phi4-mini:3.8b	2.32GB	3.8B	674	2436	ótimo 4p
qwen3:0.6b	0.49GB	751.63M	498	572	simples
qwen3:1.7b	1.27GB	2.0B	1018	715	simples
qwen3:4b	2.33GB	4.0B	859	1019	bom
qwen3.5:2b	2.55GB	2.3B	1385	1516	bom
qwen3.5:4b	3.16GB	4.7B	1380	1117	bom
llama3:8b	4.34GB	8.0B	1041	780	ok
llama3.2:3b	1.88GB	3.2B	470	1219	bom
mistral:7b	4.07GB	7.2B	1060	1969	bom estruturado
llm2.5-thinking:1.2b	0.68GB	1.2B	287	557	simples think
gemma3:270m	0.27GB	268.10M	32	1855	inglês
llama2:7b	3.56GB	7B	1233	2137	inglês

RAG e Embedding

- RAG (Retrieval-Augmented Generation) e *Embedding* permite que você consulte seus próprios documentos de forma privada e precisa.
- Convertendo seus textos em vetores numéricos e utilizando um modelo de linguagem para responder perguntas com base nesse contexto.

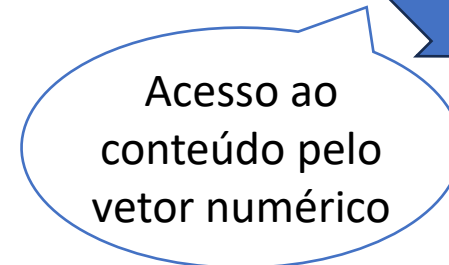
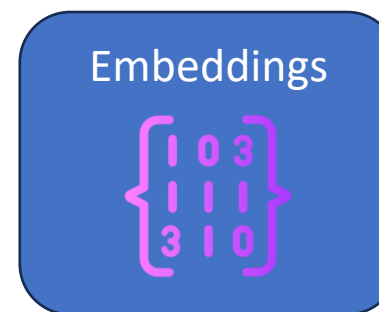


Carga Inicial



Modelos de IA

Recuperação



Banco de dados vetorial



EmbeddingGemma-300m
Google's new efficient embedding model

EmbeddingGemma

- Desenvolvido pelo Google DeepMind
- Treinado para converter texto em representações numéricas
- Capturam o significado semântico
- Leve e código aberto
- Projetado para dispositivos locais

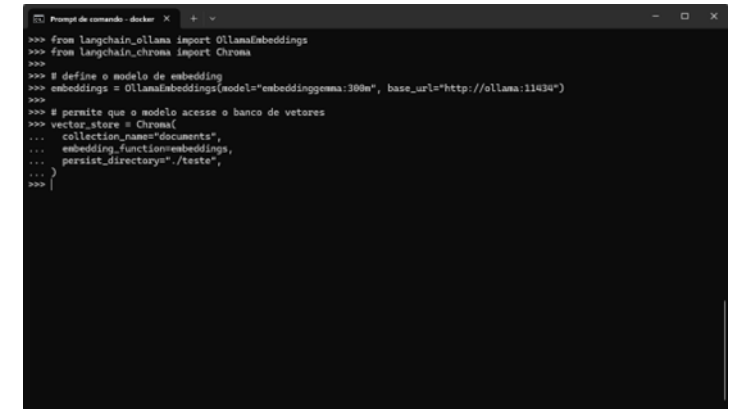
MTEB (Multilingual, v2)

Model*	Size	Mean (Task)	Retrieval	Classification	Clustering
EmbeddingGemma	308M	61.15	62.49	60.90	51.17
granite-embedding-278m-multilingual	278M	53.74	52.20	54.09	41.41
gte-multilingual-base	305M	58.24	56.50	57.17	44.33
multilingual-e5-large	560M	58.55	54.08	59.43	41.70
bge-m3	568M	59.56	54.60	60.35	40.88
jina-embeddings-v3	572M	58.37	55.76	58.77	45.65
Qwen-Embedding-0.6B	595M	64.34	64.65	66.83	52.33

*GENERAL-PURPOSE OPEN EMBEDDING MODELS

```
> from langchain_ollama import OllamaEmbeddings
> from langchain_chroma import Chroma
>
> embeddings =
> OllamaEmbeddings(model="embeddinggemma:300m",
> base_url="http://ollama:11434")
>
> vector_store = Chroma(
>     collection_name="documents",
>     embedding_function=embeddings,
>     persist_directory="./teste",
> )
```

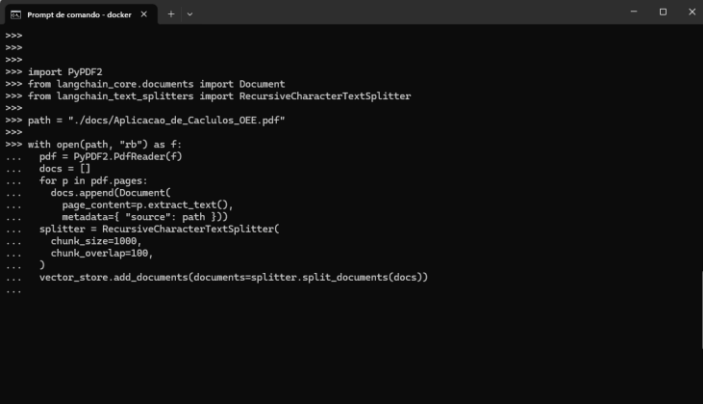
Teste Rag e Embeddings



```
Pront de comando - docker x + - x
>>> from langchain_ollama import OllamaEmbeddings
>>> from langchain_chroma import Chroma
>>>
>>> # define o modelo de embedding
>>> embeddings = OllamaEmbeddings(model="embeddinggemma:300m", base_url="http://ollama:11434")
>>>
>>> # permite que o modelo acesse o banco de vetores
>>> vector_store = Chroma(
...     collection_name="documents",
...     embedding_function=embeddings,
...     persist_directory="./teste",
... )
>>>
```

```
> import PyPDF2
> from langchain_core.documents import Document
> from langchain_text_splitters import
  RecursiveCharacterTextSplitter
> path = "./docs/Aplicacao_de_Caclulos_OEE.pdf"
> with open(path, "rb") as f:
>     pdf = PyPDF2.PdfReader(f)
>     docs = []
>     for p in pdf.pages:
>         docs.append(Document(
>             page_content=p.extract_text(),
>             metadata={ "source": path }))
>     splitter = RecursiveCharacterTextSplitter(
>         chunk_size=1000,
>         chunk_overlap=100)
>     vector_store.add_documents(documents=splitter.split_
documents(docs))
```

Teste Rag e Embeddings



```
Prompt de comando - docker x + -
>>>
>>>
>>> import PyPDF2
>>> from langchain_core.documents import Document
>>> from langchain_text_splitters import RecursiveCharacterTextSplitter
>>> path = "./docs/Aplicacao_de_Caclulos_OEE.pdf"
>>>
>>> with open(path, "rb") as f:
...     pdf = PyPDF2.PdfReader(f)
...     docs = []
...     for p in pdf.pages:
...         docs.append(Document(
...             page_content=p.extract_text(),
...             metadata={ "source": path }))
...     splitter = RecursiveCharacterTextSplitter(
...         chunk_size=1000,
...         chunk_overlap=100,
...     )
...     vector_store.add_documents(documents=splitter.split_documents(docs))
>>>
```

```

> query = "o que é o cálculo o.e.e?"
> result = vector_store.similarity_search_with_score(query,
k=5)
> for doc, score in result:
>     print(f"content:
{doc.page_content.strip()[:100].replace("\n", " ")}...")
>     print(f"metadata: {doc.metadata}", "\nscore:", score,
"\n")

> content: Aplicação de Cálculos O.E.E.
Revisão 30/04/2025...

> metadata: {'source':
'./docs/Aplicacao_de_Caclulos_OEE.pdf'}

> score: 0.9016335010528564

> content: Página : 1
, o exemplo d e como é realizad o os cálculos do O.E....
> metadata: {'source':
'./docs/Aplicacao_de_Caclulos_OEE.pdf'}

> score: 1.02092707157135

> ...

```

Teste Rag e Embeddings

```

Prompt de comando - docker x + -
>>> query = "o que é o cálculo o.e.e?"
>>>
>>> # lista de contextos de acordo com a similaridade
>>> result = vector_store.similarity_search_with_score(query, k=5)
>>>
>>> # exibição
>>> for doc, score in result:
...     print(f"content: {doc.page_content.strip()[:100].replace("\n", " ")}...")
...     print(f"metadata: {doc.metadata}", "\nscore:", score, "\n")
...
content: Aplicação de Cálculos O.E.E. Revisão 30/04/2025...
metadata: {'source': './docs/Aplicacao_de_Caclulos_OEE.pdf'}
score: 0.9016335010528564

content: Página : 1 Segue abaixo, o exemplo d e como é realizad o os cálculos do O.E....
metadata: {'source': './docs/Aplicacao_de_Caclulos_OEE.pdf'}
score: 1.02092707157135

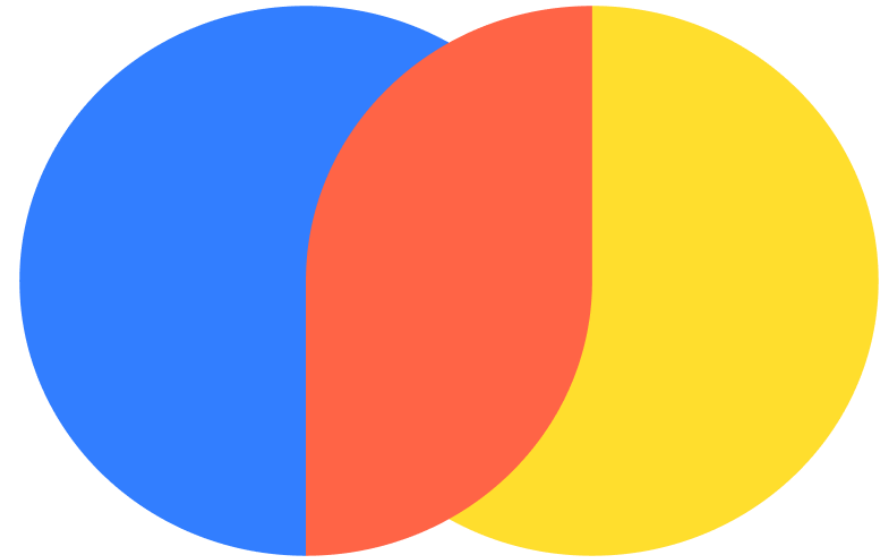
content: produtos dentro das especificações. Q / (M + N) Formula...
metadata: {'source': './docs/Aplicacao_de_Caclulos_OEE.pdf'}
score: 1.3206607103347778

>>>

```


ChromaDB

- Banco de dados vetorial de código aberto
- Projetado para armazenar, gerenciar e recuperar *embeddings* de vetore
- Amplamente utilizado em aplicações de Inteligência Artificial (IA) generativa
- Foco em busca semântica e RAG

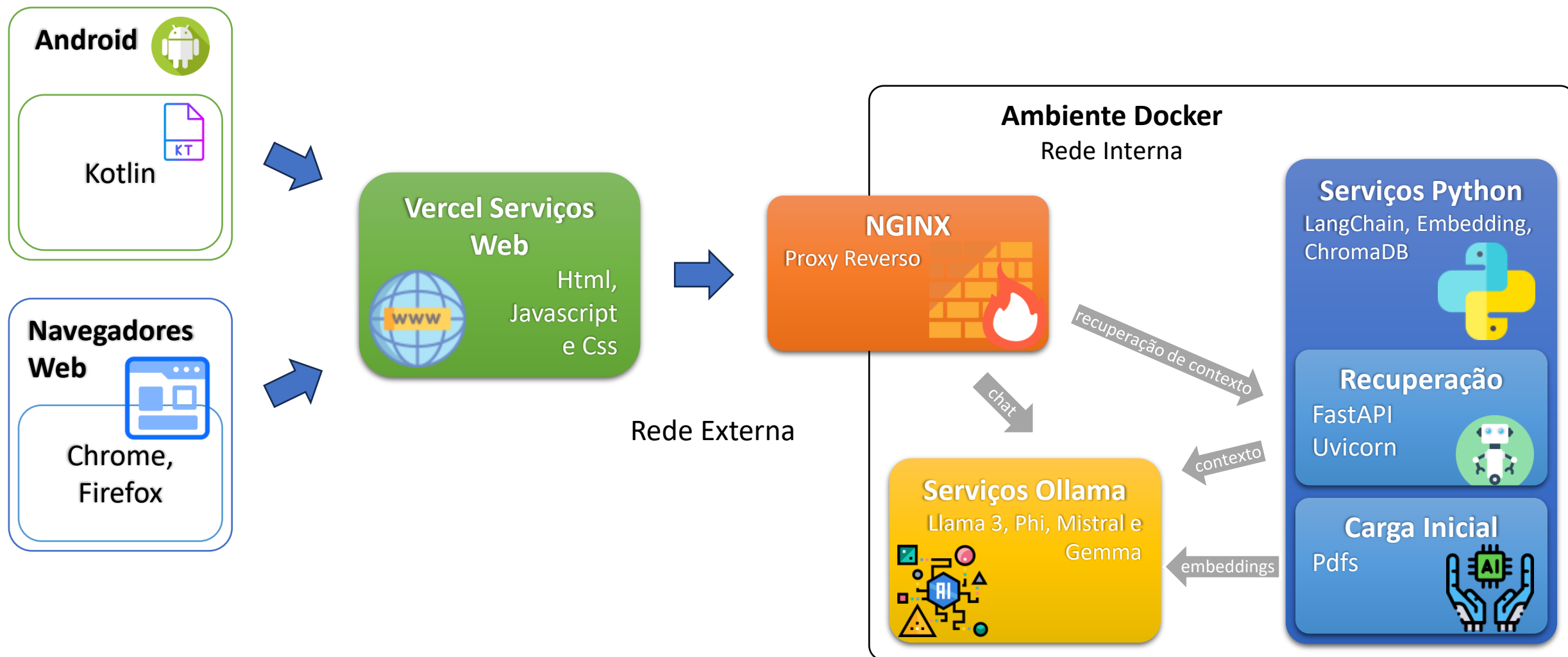


Outros bancos de dados vetoriais

- **Pinecone**: facilidade de uso e escalabilidade em nuvem
- **Milvus**: grandes conjuntos de dados
- **Qdrant**: escrito em Rust, alta performance e recursos avançados de filtragem
- **Weaviate**: nativo de nuvem com forte capacidade de busca híbrida
- **pgvector**: extensão para PostgreSQL
- **Faiss**: pelo Meta para busca e agrupamento de vetores densos
- **Redis**: em memória
- **OpenSearch**: pesquisa de texto completo com similaridade semântica
- **LanceDB**: leve e facilidade de integração em pipelines



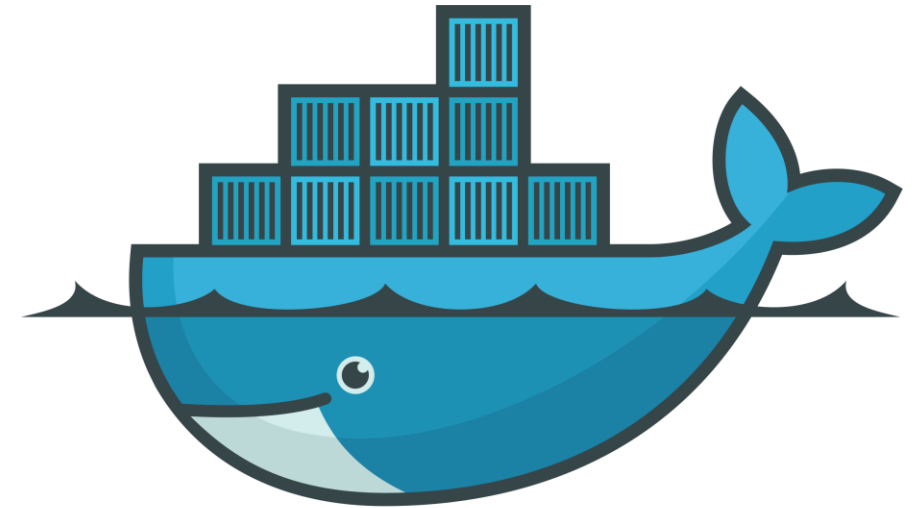
Diagrama da arquitetura do sistema



Docker

Plataforma de código aberto que automatiza a criação, implantação e execução de aplicações em contêineres

Container é uma unidade de software leve, autossuficiente e isolada que empacota o código de uma aplicação junto com todas as suas dependências



docker

Vercel

- Plataforma de computação em nuvem focada no desenvolvedor
- Foca em hospedar sites e aplicativos web
- Destaca-se na velocidade e integração nativa com o Next.js
- Características: foco no front-end, conexão com Git, e Ambiente Serverless
- Vantagens: implantação rápida, visualização de pull requests (cria links únicos de testes para cada alteração no código antes de ir para o ar), e domínios personalizados



FastAPI

- Framework web moderno, de alto desempenho e rápido para construir APIs



Uvicorn

- Servidor web ASGI (Asynchronous Server Gateway Interface)
- Altíssima performance para Python, ideal para frameworks assíncronos
- Projetado para lidar com conexões HTTP/1.1 e WebSockets



NGINX

- Software de servidor web de alto desempenho, código aberto e gratuito, amplamente utilizado como proxy reverso, balanceador de carga, cache HTTP e servidor de e-mail
- Arquitetura assíncrona orientada a eventos
- Lida com milhares de conexões simultâneas consumindo pouca memória



Kotlin

- Linguagem de programação moderna, concisa e de código aberto
- Permite criar aplicativos para Android
- Desenvolvida pela JetBrains
- Interoperável com Java
- Desenvolvimento Android



LangChain

- Framework open-source
- Simplifica a criação de aplicações com LLM
- Facilita conexão com fontes de dados externas, memória e ferramentas
- Encadeia funcionalidades



Referências

- <https://aws.amazon.com/what-is/retrieval-augmented-generation/>
- <https://www.hostinger.com/br/tutoriais/o-que-e-ollama>
- <https://triggo.ai/blog/o-que-e-retrieval-augmented-generation/>
- <https://pt.community.intersystems.com/post/modelos-llm-e-aplica%C3%A7%C3%B5es-rag-passo-passo-parte-ii-criando-o-contexto>
- <https://www.ibm.com/br-pt/think/topics/langchain>
- <https://ai.google.dev/gemma/docs/embeddinggemma?hl=pt-br>
- <https://ollama.com/library/embeddinggemma>
- <https://www.datacamp.com/pt/tutorial/chromadb-tutorial-step-by-step-guide>
- <https://www.instaclustr.com/education/vector-database/top-10-open-source-vector-databases/>
- <https://aws.amazon.com/pt/compare/the-difference-between-docker-images-and-containers/>
- <https://www.ibm.com/br-pt/think/topics/docker>
- <https://cloud.google.com/use-cases/langchain?hl=pt-BR>
- <https://www.f5.com/glossary/nginx>
- <https://kotlinlang.org/>
- <https://dev.to/sucodelarangela/como-colocar-um-projeto-no-ar-com-vercel-2m9d>